

PHPMySQL UNIT - 2

Made from the IPU recommended textbook **Programming PHP, First Edition, by Rasmus Lerdorf and Kevin Tatroe (O'Reilly).** and **PHP, MySQL and Apache by Julie C Meloni. Pearson Education**

Unit 2: Arrays and Form Handling

1. Arrays

Arrays are a fundamental data structure in PHP. They are used to store multiple values in a single variable, organized as an ordered collection of key-value pairs.

1.1 Anatomy of an Array

- **Core Concept:** An array is a map, a data structure that maps keys to values.
- **Key:** The identifier used to access an element. It can be an integer (for indexed arrays) or a string (for associative arrays). Keys must be unique within an array.
- **Value:** The data stored in the array element, which can be of any data type (integer, string, another array, object, etc.).
- **Internal Structure:** PHP internally stores all arrays as **ordered maps**. This means that not only are the key-value associations preserved, but the insertion order of the elements is also maintained.

1.2 Creating Index-Based and Associative Arrays

array

NUMERIC

NAME

KEYWORD

ARRAY KEYS

[0][1][2][3][4][5]

ARRAY VALUES

\$name

=

array(

4,3,5,2,1,0

)

;

Numeric arrays have **numeric keys**. The numeric keys are automatically created for all values specified in an array.

In PHP error reporting, undefined numeric keys are referred to as offset.

Array keys are accessed using the [] square brackets.

ASSOCIATIVE

KEY NAME

KEY VALUE

\$name

=

array(

'Sam' => 23,

'Bob' => 35,

'Jim' => 42

)

;

ROCKET

Associative arrays have **named keys**. The key names are placed inside single or double quotes and the => (rocket) is used to assign key => value pairs.

In PHP error reporting, undefined named keys are referred to as index.

There are two primary types of arrays in PHP, distinguished by their key types.

Basis	Indexed Array	Associative Array
Key Type	Integer keys, usually sequential and starting from 0.	String keys, which are descriptive names for the values.
Use Case	Used for creating simple, ordered lists of items like a list of names, numbers, or tasks.	Used for storing data with named properties, like a person's details (name, age, city).
Creation	Keys are automatically assigned if not specified.	Keys must be explicitly defined using the <code>=></code> operator.

- **Creating an Indexed Array:**

- You can use the `array()` construct or, from PHP 5.4+, the short array syntax `[]`. If keys are omitted, they are automatically indexed starting from 0.

```
<?php
// Using the array() construct
$fruits = array("Apple", "Banana", "Cherry");

// Using short array syntax
$cars = ["Volvo", "BMW", "Toyota"];

// Adding elements one by one
$colors = [];
$colors[] = "Red";    // Index 0
$colors[] = "Green";  // Index 1
$colors[] = "Blue";   // Index 2

print_r($fruits);
?>
```

- **Creating an Associative Array:**

- Use the `=>` operator to associate a key with a value.

```
<?php
// Using the array() construct
$person = array(
    "firstName" => "John",
    "lastName"  => "Doe",
    "age"       => 30
);

// Using short array syntax
$user = [
    "username" => "johndoe",
```

```

        "email" => "john.doe@example.com"
    ];

    // Adding elements one by one
    $book = [];
    $book['title'] = "Programming PHP";
    $book['author'] = "Rasmus Lerdorf";

    print_r($person);
?>

```

1.3 Accessing Array Elements

- You can access an array element by referring to its key inside square brackets `[]`.
- It is considered good practice to always enclose string keys in quotes (`'` or `"`).

```

<?php
// Accessing an indexed array element
$cars = ["Volvo", "BMW", "Toyota"];
echo "I like " . $cars[1]; // Outputs: I like BMW

echo "<br>";

// Accessing an associative array element
$person = array("firstName" => "John", "lastName" => "Doe");
echo "His name is " . $person['firstName']; // Outputs: His name is John
?>

```

1.4 Looping with Arrays

- **Looping with an Index-Based Array using `for`:**
 - Best suited for arrays with a continuous sequence of integer keys.
 - Use the `count()` function to determine the number of elements.

```

<?php
$colors = ["Red", "Green", "Blue"];
$arrlength = count($colors);

for ($x = 0; $x < $arrlength; $x++) {
    echo $colors[$x];
    echo "<br>";
}
?>

```

- **Looping with an Associative Array using `foreach`:**

- The `foreach` loop is the modern and preferred way to iterate over any array (indexed or associative).
- It works on a copy of the array, ensuring the original is not affected by changes inside the loop.

```
<?php
// Looping to get only values
$fruits = array("Apple", "Banana", "Cherry");
foreach ($fruits as $value) {
    echo "$value <br>";
}

echo "<hr>";

// Looping to get keys and values
$person = array("firstName" => "John", "lastName" => "Doe", "age" => 30);
foreach ($person as $key => $value) {
    echo "$key: $value <br>";
}
?>
```

- **Looping with `each()` and `while()`:**

- This is an older method but useful for understanding array pointers.
- `each()` returns the current key/value pair from an array and advances the internal array pointer.
- It is often used with `list()` to assign the key and value to variables.
- **Note:** `each()` is deprecated as of PHP 7.2.0 and removed in PHP 8.0.0. `foreach` should always be preferred.

```
// This code is for educational purposes and will not run on PHP 8+
$person = array("firstName" => "John", "lastName" => "Doe");
reset($person); // Reset the internal pointer to the start
while (list($key, $value) = each($person)) {
    echo "$key: $value <br>";
}
```

1.5 Some Useful Library Functions

Function	Description	Example
<code>count(\$array)</code>	Returns the number of elements in an array.	<code>count(["a", "b", "c"]); //</code> Returns 3
<code>sort(\$array)</code>	Sorts an indexed array in ascending order. Keys are re-indexed.	<code>\$a = [3,1,2]; sort(\$a); //</code> <code>\$a</code> is now [1,2,3]

Function	Description	Example
<code>rsort(\$array)</code>	Sorts an indexed array in descending order. Keys are re-indexed.	<pre>\$a = [1,3,2]; rsort(\$a); // \$a is now [3,2,1]</pre>
<code>asort(\$array)</code>	Sorts an associative array by value in ascending order, preserving key-value association.	<pre>\$a = ["b"=>3,"a"=>1]; asort(\$a); // \$a is ["a"=>1,"b"=>3]</pre>
<code>ksort(\$array)</code>	Sorts an associative array by key in ascending order, preserving key-value association.	<pre>\$a = ["b"=>1,"a"=>3]; ksort(\$a); // \$a is ["a"=>3,"b"=>1]</pre>
<code>array_push(\$array, \$val)</code>	Pushes one or more elements onto the end of an array.	<pre>\$a = [1]; array_push(\$a, 2, 3); // \$a is [1,2,3]</pre>
<code>array_pop(\$array)</code>	Pops the element off the end of an array.	<pre>\$a = [1,2,3]; array_pop(\$a); // returns 3, \$a is [1,2]</pre>
<code>array_keys(\$array)</code>	Returns all the keys of an array.	<pre>array_keys(["a"=>1, "b"=>2]); // returns ["a", "b"]</pre>
<code>in_array(\$needle, \$haystack)</code>	Checks if a value exists in an array. Returns <code>true</code> or <code>false</code> .	<pre>in_array("apple", ["apple", "banana"]); // returns true</pre>

2. Handling HTML Forms with PHP

PHP is exceptionally good at collecting data from HTML forms.

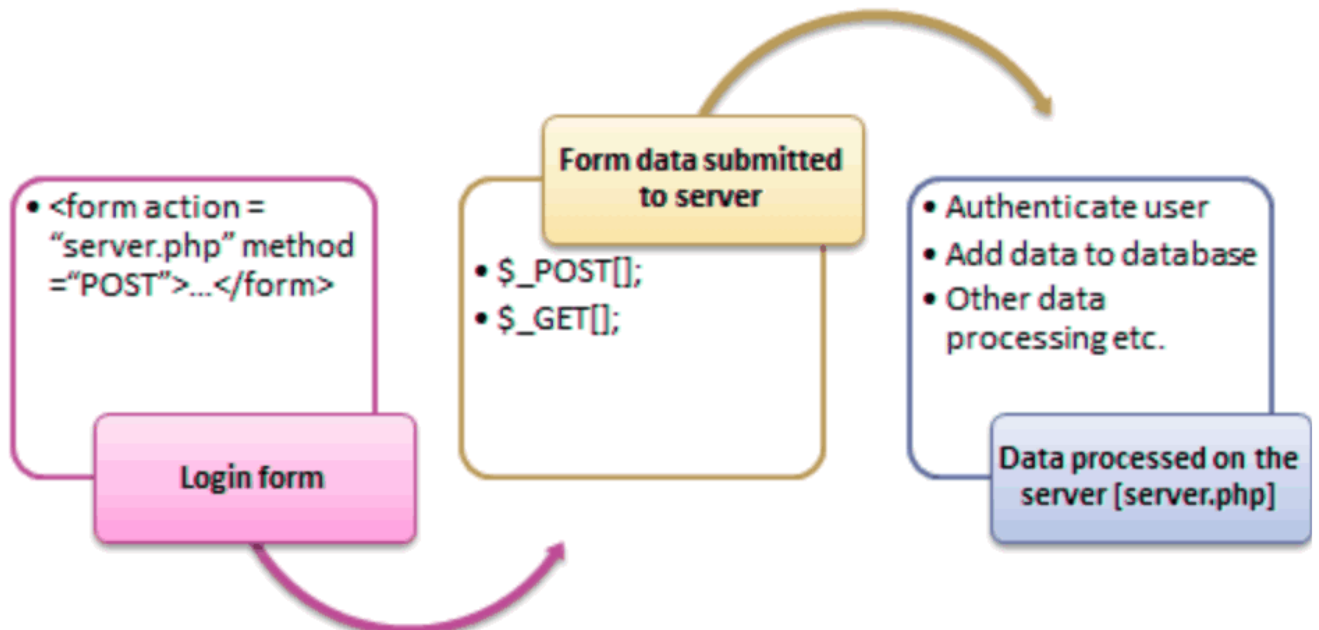
2.1 Capturing Form Data

- **HTML Form Basics:**

- An HTML `<form>` tag is used to create a form for user input.
- The `action` attribute specifies the PHP file that will process the form data.
- The `method` attribute specifies how to send the data (`GET` or `POST`).

- GET vs. POST Method

Get and Post Method in PHP



Basis	GET	POST
Visibility	Form data is visible in the URL (e.g., <code>...page.php?name=john&age=30</code>).	Form data is sent in the HTTP request body and is not visible in the URL.
Data Limit	Limited to about 2000 characters.	No restrictions on data size. Suitable for large data and file uploads.
Security	Less secure. Not suitable for sensitive data like passwords.	More secure than GET as data is not exposed in the URL.
Use Case	Search forms, bookmarkable pages.	Login forms, registration forms, file uploads, any action that modifies data on the server.

- PHP Superglobals:

- PHP provides special, built-in variables called superglobals to access form data.
- `$_GET`: An associative array of variables passed to the current script via URL parameters (GET method).
- `$_POST`: An associative array of variables passed to the current script via the HTTP POST method.
- `$_REQUEST`: An array containing the contents of `$_GET`, `$_POST`, and `$_COOKIE`.

- Example: Capturing Form Data

HTML (`user_form.html`)

```

<form action="welcome.php" method="post">
  Name: <input type="text" name="name"><br>
  Email: <input type="text" name="email"><br>

```

```
<input type="submit" value="Submit">
</form>
```

PHP (`welcome.php`)

```
<?php
// Use isset() to check if the form has been submitted
if (isset($_POST['name'])) {
    $name = $_POST['name'];
    $email = $_POST['email'];
    echo "Welcome, " . htmlspecialchars($name) . "!<br>";
    echo "Your email address is: " . htmlspecialchars($email);
} else {
    echo "Please submit the form first.";
}
?>
```

2.2 Dealing with Multi-value Fields

- Some form elements like checkboxes or multiple-select lists can return more than one value.
- To capture these as an array in PHP, the `name` attribute of the form element must end with square brackets `[]`.

- **Example: Checkboxes**

HTML

```
<form action="process.php" method="post">
    What languages do you know?<br>
    <input type="checkbox" name="languages[]" value="PHP"> PHP<br>
    <input type="checkbox" name="languages[]" value="JavaScript">
JavaScript<br>
    <input type="checkbox" name="languages[]" value="Python"> Python<br>
    <input type="submit">
</form>
```

PHP (`process.php`)

```
<?php
if (isset($_POST['languages'])) {
    $langs = $_POST['languages'];
    echo "You know the following languages:<br>";
    // The data is an array, so we can loop through it
    foreach ($langs as $language) {
        echo "- " . htmlspecialchars($language) . "<br>";
    }
} else {
    echo "You didn't select any languages.";
}
```

```
}  
?>
```

2.3 Generating File Uploaded Form

- **HTML Form Requirements:**

1. The method **must** be `POST`.
2. The form needs an `enctype="multipart/form-data"` attribute. This specifies that the form will be sending binary data.
3. Use `<input type="file">` to create the file selection field.

- **PHP Handling (`$_FILES` Superglobal):**

- PHP puts information about uploaded files into the `$_FILES` associative array.
- `$_FILES['input_name']['name']`: The original name of the file on the client machine.
- `$_FILES['input_name']['type']`: The MIME type of the file (e.g., `image/jpeg`).
- `$_FILES['input_name']['tmp_name']`: The temporary filename where the uploaded file is stored on the server. **This is the file you must work with.**
- `$_FILES['input_name']['error']`: An error code associated with the file upload. `0` means no error.
- `$_FILES['input_name']['size']`: The size of the uploaded file in bytes.

- **Security and Processing:**

- Use `is_uploaded_file()` to verify that the file was indeed uploaded via HTTP POST.
- Use `move_uploaded_file()` to move the file from the temporary directory to a permanent location. This is the secure way to handle uploaded files.

- **Example: File Upload**

HTML

```
<form action="upload.php" method="post" enctype="multipart/form-data">  
    Select image to upload:  
    <input type="file" name="profile_picture">  
    <input type="submit" value="Upload Image">  
</form>
```

PHP (`upload.php`)

```
<?php  
$target_dir = "uploads/";  
$target_file = $target_dir . basename($_FILES["profile_picture"]  
["name"]);  
  
// Check if a file was uploaded without errors  
if (isset($_FILES["profile_picture"]) && $_FILES["profile_picture"]  
["error"] == 0) {
```



```

    // Use move_uploaded_file for security
    if (move_uploaded_file($_FILES["profile_picture"]["tmp_name"],
    $target_file)) {
        echo "The file ".
    htmlspecialchars(basename($_FILES["profile_picture"]["name"])) ." has
    been uploaded.";
    } else {
        echo "Sorry, there was an error moving your file.";
    }
} else {
    echo "Error: No file uploaded or an upload error occurred.";
}
?>

```

2.4 Redirecting a Form After Submission

- **Purpose:** To prevent users from accidentally re-submitting a form by refreshing the page after a successful submission (the Post/Redirect/Get pattern).
- **Method:** Use the `header()` function to send a `Location` header to the browser, telling it to go to a different page.
- **Crucial Rule:** The `header()` function **must** be called before any HTML or text output is sent to the browser.
- It is best practice to call `exit()` immediately after `header()` to stop the rest of the script from executing.

```

<?php
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    // 1. Process the form data (e.g., save to a database)
    // ...

    // 2. Redirect to a "thank you" or success page
    header("Location: thank_you.php");
    exit(); // Stop script execution
}
?>

```

```

<!-- HTML form goes here -->
<form method="post" action="<?php echo
htmlspecialchars($_SERVER["PHP_SELF"]);?>">
    <!-- Form fields -->
    <input type="submit" value="Submit">
</form>

```